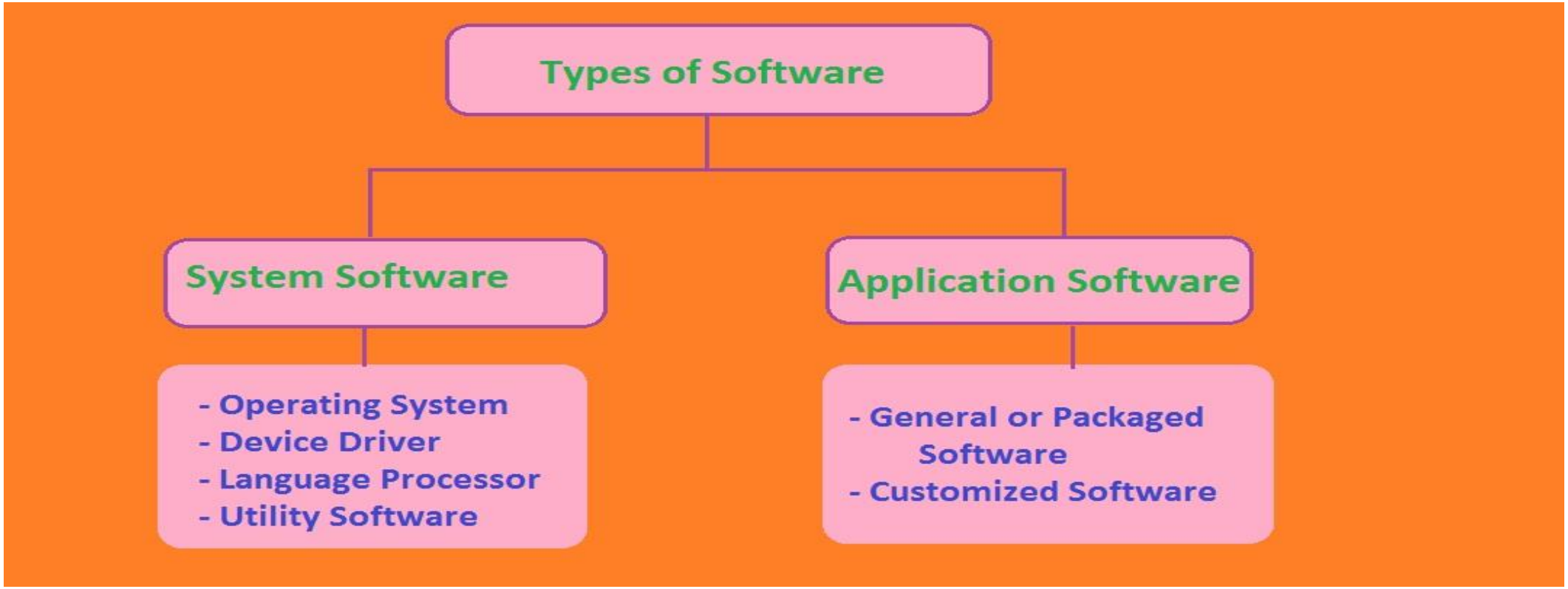
Unit I
Introduction

What is software?

- Software is a set of instructions or programs that tells a computer what to do.
- Software is collection of computer programs and associated documentation such as requirements, design models and user manuals.
- Software products may be developed for a particular customer or may be developed for a general market.
- New software can be created by developing new programs, configuring generic software systems or reusing existing software.
- Software products may be:
  - **Generic** - developed to be sold to a range of different customers e.g. PC software such as Excel or Word.
  - **Bespoke (custom)** - developed for a single customer according to their specification.

Types of Software

➢Software can be categorized according to what it is designed to accomplish.



System Software

- The type of software that supports and manages the computer resources and operations of the computer system is known as system software.
- It creates a communication link between the computer and user.
- It also provides and maintains an environment or platform for running application software.
- Futher divided into four sub types:

a. **Operating System:** Controls and coordinates overall operations of the computer system
b. **Device Driver:** Controls the devices attached to the computer system.
c. **Language Processor:** Language processor is a type of system software that translates the programming language code into machine code. **Assembler, Interpreter and Compiler.**
d. **Utility Software:** software used for maintenance of computer system like Antivirus, Network utilities, Screen savers, File managers, Disk defragmenters, Disk cleaners etc.

System Software

- The type of software that supports and manages the computer resources and operations of the computer system is known as system software.
- It creates a communication link between the computer and user.
- It also provides and maintains an environment or platform for running application software.
- Futher divided into four sub types:

a. Operating System: Controls and coordinates overall operations of the computer system
b. Device Driver: Controls the devices attached to the computer system.
c. Language Processor: Language processor is a type of system software that translates the programming language code into machine code.
d. Utility Software-software used for maintenance of computer system like Antivirus, Network utilities, Screen savers, File managers, Disk defragmenters, Disk cleaners etc.

Functions of System Software

1. Monitoring the use of all the hardware components
2. Providing an easy and efficient interaction between the computer and user.
3. Communication with the peripheral devices such as a scanner, printer, webcam etc.
4. Managing files and folders
5. Providing security
6. Providing platform for application software

Application Software

- Application software is a type of software that is designed to solve a specific problem of the user or to perform the specific task.
- The application software can be sub-divided into two types.

a. **Tailored or customized Software:** Sometimes companies, organization or an individual needs specific types of **software to solve specific problems**, so they ask the software development companies or individual programmers to develop a custom software that meets the requirement to solve their problem. For instance, Payroll system, Result processing, Library management system, Banking system, School management system etc. are the example of customized software.

b. **General or Packaged Software:** General software are those types of software that are made by software companies **to solve the general problems of people**. Some examples of these types of software are MS-Excel, MS-Access, Photoshop, Dreamweaver, Notepad, etc.

Freeware, Shareware and Open-Source Software

1. **Freeware Software:** Freeware software is available without any cost. Any user can download it from the internet and use it without paying any fee. However, freeware does not provide any liberty for modifying the software or charging a fee for its distribution. Example: Adobe Reader, Team Viewer, Skype, etc.
2. **Shareware Software:** It is a software that is freely distributed to users on a trial basis. It usually comes with a time limit and when the time limit expires, the user is asked to pay for the continued services. Example: Adobe Acrobat, PHP Debugger, etc.
3. **Open-Source Software:** These kinds of software are available to users with the source code which means that a user can freely distribute and modify the software and add additional features to the software. Open-Source software can either be free or chargeable. Example: Apache Web Server, Thunderbird, Mozilla Firefox, etc

Software Engineering

- Software engineering is an **engineering discipline** that is concerned with **all aspects of software production** from the early stages of system specification to maintaining the system after it has gone into use.
  - **Engineering Discipline:** Using appropriate theories and methods to solve problems bearing in mind organizational and financial constraints.
  - **All Aspects of Software Production:** Not just technical process of development, other aspects such as project management and development tools, methods for software production must also be considered.
- The outcome of software engineering is an efficient and reliable software product.



Fundamental Software Engineering Activities (High Level View)

- Four fundamental activities that are part of all software development processes:

a. Software Specification
b. Software Development
c. Software Validation
d. Software Evolution

Software Engineer vs Software Developer

Software Engineer	Software Developer
A software engineer is a professional who applies the principles of software engineering for designing, development, maintenance, testing, and evaluation of computer software.	Software developers are professionals who build software which runs across various types of computer. They write code from scratch.
Applies Engineering principles and knowledge to the creation of software product.	Engineering principles not involved.
Software Engineer tends to solve issues on a much larger scale.	Software Developers solve on a limited scale.

Software Project

- A complete procedure of software development from gathering requirements to testing and maintenance carried out in a specified period of time.
- A project is defined as a sequence of tasks that must be completed to attain a certain outcome.
- A Software Project is the complete procedure of software development from requirement gathering to testing and maintenance, carried out according to the execution methodologies, in a specified period of time to achieve intended software product.

Is Software Project and Software Product the same?

Software Project-Key Characteristics

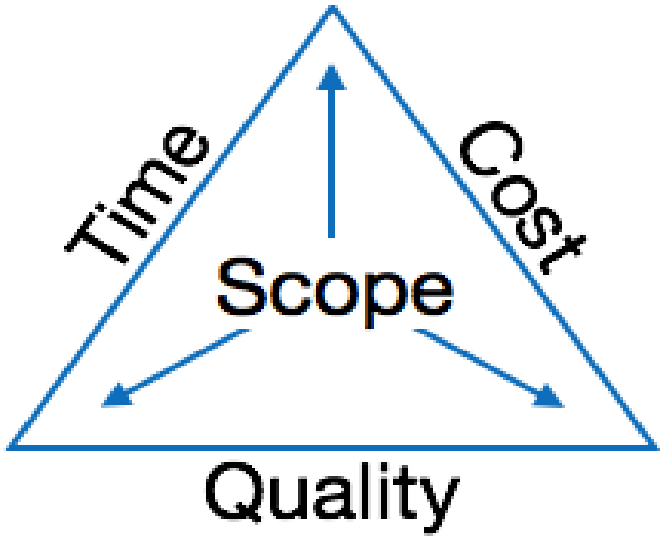
1. **Defined Objectives**
2. **Unique Deliverables**
3. **Temporary Nature**
4. **Resource Constraints**
5. **Risk and Uncertainty**

Software Project-Major Components

1. **Requirements**
2. **Project Team**
3. **Development Process**
4. **Technology Stack**
5. **Quality Assurance**

Software Project Management (SPM)

- Project Management disciplines for Software Projects
- Software Project Management focuses on planning, executing, monitoring, controlling, and closing software development projects.
- Software in project management is dedicated to the planning, scheduling, resource allocation, execution, tracking, and delivery of software projects.
- Software Project Management is essential to incorporate user requirements along with budget and time constraints.



Management Spectrum/4 P's of Software Project Management

- Project Management Spectrum describes the management of a software project or how to make a project successful.
- It focuses on the 4 P's:
  1. People
  2. Process
  3. Project
  4. Product

Software Project vs Software Product

Software Project	Software Product
It comprises the steps involved in making a software before it is actually available to the market.	It is the manufacture/result of the project for users.
The main goal of a project is to form a new product that has not already been made.	The main goal of the product is to solve a specific problem for which the product was built.
Project is undertaken to form a new software.	Product is the final production of the project.
Projects have time constraints.	Products are result of project that do not have time constraints.
It has some cost barriers as the enterprise sets a fixed budget for the project.	It has no cost barrier as it is continued to be developed only as long as the distributing enterprise is profitable.
Software Project is handled by Software Project Manager.	Software Product is handled by Software Product Manager.

Participants and roles in Software Projects

- Participants in a software project are the individuals, groups, or organizations involved in the planning, development, management, use, and maintenance of the software system.
  1. Stakeholders
  2. Project Manager
  3. System Analyst
  4. Software Architect
  5. Software Developers
  6. QA Engineers/Testers
  7. Database Administrator
  8. DevOps Engineers
  9. End Users
  10. Maintenance Team

Participants and roles in Software Projects-Stakeholders

- Stakeholders are people or organizations that have an interest in the project's outcome.
- **Responsibilities**
  - Define business goals.
  - Provide project requirements.
  - Review project progress.
  - Approve deliverables.
  - Evaluate project success.
- **Examples**
  - Clients/Customers/End users
  - Sponsors
  - Management

Participants and roles in Software Projects-Project Managers

- The Project Manager oversees the entire software project and ensures that objectives are achieved within time, budget, and quality constraints.

Responsibilities

- Project planning and scheduling.
- Resource allocation.
- Risk management.
- Team coordination.
- Budget control.
- Communication with stakeholders.

Skills Required

- Leadership
- Communication
- Problem-solving
- Decision-making
- Project management methodologies

Participants and roles in Software Projects-System Analyst

- The System Analyst acts as a bridge between business users and the technical team.

Responsibilities

- Gather and analyze requirements.
- Study existing systems.
- Prepare requirement specifications.
- Recommend system solutions.
- Assist in system design.

Skills Required

- Analytical thinking
- Communication skills
- Business process understanding
- Documentation skills

Participants and roles in Software Projects-Software Architect

➢The Software Architect designs the overall structure of the software system.

Responsibilities

- Define system architecture.
- Select technologies and frameworks.
- Ensure scalability and security.
- Establish coding standards.
- Guide development teams.

Skills Required

- System design
- Problem Solving
- Technical expertise

Participants and roles in Software Projects-Software Developers

➢Developers write the code that implements the system requirements.

Responsibilities

- Develop software modules.
- Write and maintain code.
- Debug and fix issues.
- Perform unit testing.
- Integrate components.

Skills Required

- Programming languages
- Database management
- Framework knowledge
- Testing techniques
- Concept of deployment

Participants and roles in Software Projects-QA Engineers/Testers

- QA Engineers ensure that the software meets quality standards and functions correctly.

Responsibilities

- Prepare test plans.
- Execute test cases.
- Identify defects.
- Verify bug fixes.
- Perform performance and security testing.

Skills Required

- Testing tools
- Analytical skills
- Attention to detail
- Documentation

Participants and roles in Software Projects-Database Administrator

➢The Database Administrator(DBA) manages the project's databases.

Responsibilities

- Database design.
- Data security.
- Backup and recovery.
- Performance optimization.
- Database maintenance.

Skills Required

- SQL
- Database management systems
- Security management
- Data modeling

Participants and roles in Software Projects-DevOps Engineer

- DevOps engineering bridges the gap between software development and IT operations. It focuses on automating workflows, deploying updates faster, and ensuring system reliability.

Responsibilities

- Continuous Integration (CI).
- Continuous Deployment (CD).
- Infrastructure management.
- Monitoring and automation.
- System reliability improvement.

Skills Required

- Cloud platforms
- Automation tools
- Scripting
- System administration

Participants and roles in Software Projects-End Users

➢End users are the people who will use the software after deployment.

Responsibilities

- Provide requirements and feedback.
- Participate in testing.
- Report issues.
- Evaluate system usability.

System and Models

- **System:** An organized set of components that interact with one another to process inputs, produce outputs, and achieve predefined objectives.
- **Model:** A model is an abstract representation of a system that highlights important aspects while ignoring unnecessary details. It is an abstract model of the reality
- System Models can be categorized as:
  1. **Context Model:** Context Models are high-level visual representations that show the interactions between a system being developed and its external entities, such as users, other systems, or processes.
  2. **Interaction Models:** A representation of the interactions that occur between users, software components, or external systems during the execution of a process or function. e.g. Use case Diagram, Sequence Diagram, etc.
  3. **Structural Models:** Shows system organization in terms of components, e.g. class diagram, object diagram, etc.
  4. **Behavioral Models:** Shows dynamic behavior or run-time behavior. e.g. sequence diagram, activity diagram, state-machine diagram, etc.
  5. **Data Model:** Represents how data is organized, stored, related, and managed. e.g. ER-Diagram, Schema Diagram, etc.

Work Products

- These are the **tangible outputs** produced during the software development life cycle (SDLC).
- A work product is any **deliverable artifact** produced during software development that contributes to building, verifying, or maintaining a software system.
- Examples:
  - SRS Document during requirement engineering
  - Architectural Design, ER-Diagram. Class Diagram, etc during design phse
  - Source Codes, APIs, Modules, etc during Implementation Phase
  - Test Plans, Test Scripts during Testing phase, etc.

Activities, Tasks and Resources

- Activities: An activity is a collection of related tasks performed to achieve a specific objective in the SDLC.
- Task: A **task** is a **small, specific unit of work** that is part of an activity. Tasks are detailed actions assigned to individuals or teams.
- Resources: **Resources** are the **inputs required to perform activities and tasks** in a software project. e.g human resource, hardware and software resources, data resources, financial resources, etc.

Software Processes

- A software process (also knows as software methodology) is a **set of related activities that leads to the production of the software.**
- These activities may involve the **development of the software from the scratch, or, modifying an existing system.**
- A structured set of activities required to develop a software system are:
  1. Planning and Analysis
  2. Design
  3. Implementation/Coding
  4. Testing and Integration
  5. Deployment
  6. Maintenance

Software Process Model

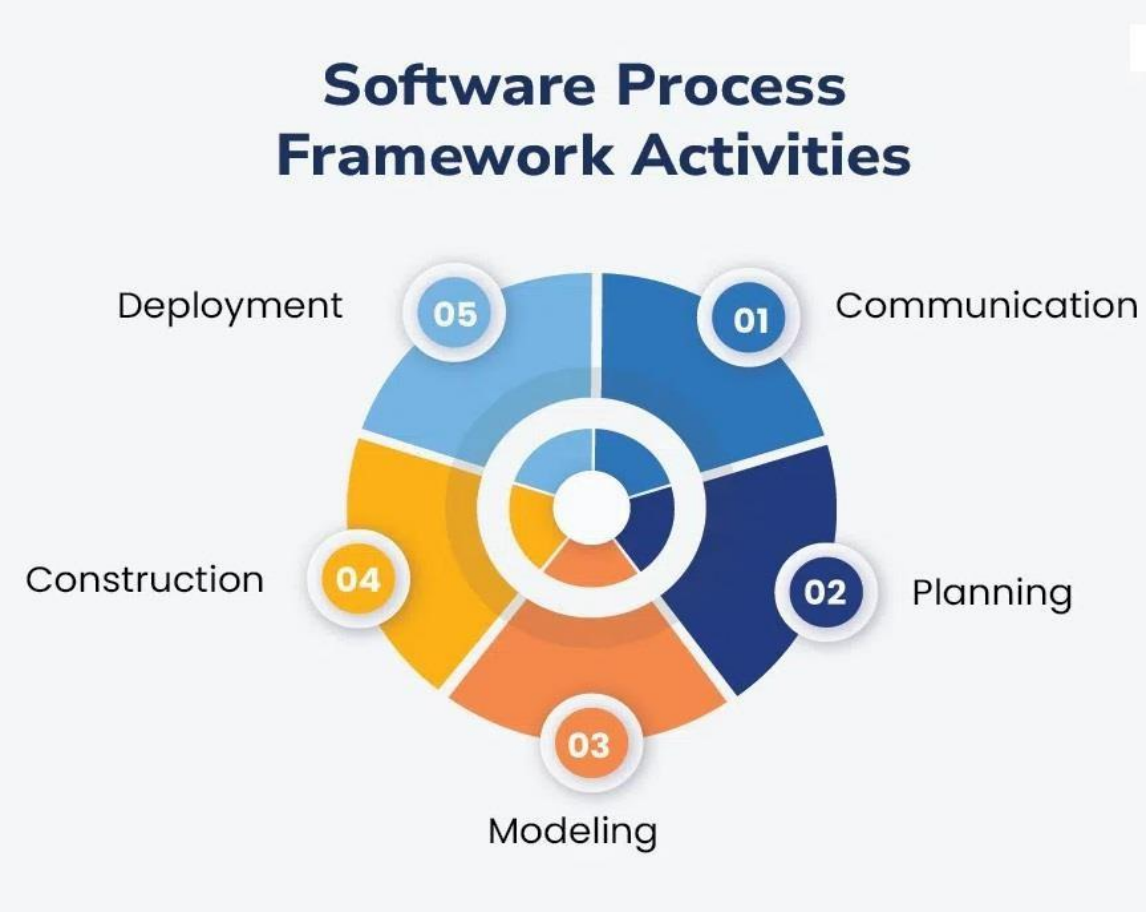
- A software process model is a simplified representation of a software process.
- A software process model is an abstraction of the software development process.
- A process model for software engineering is chosen based on:
  - the nature of project and application
  - The methods and tools to be used
  - The controls and deliverables that are required
- Process model may include:
  - Activities which are part of software process
  - Software product
  - Role of people involved in software engineering
- The goal of software process model is to **provide guidance for systematically coordinating and controlling the tasks that must be performed in order to achieve end products and product objectives.**

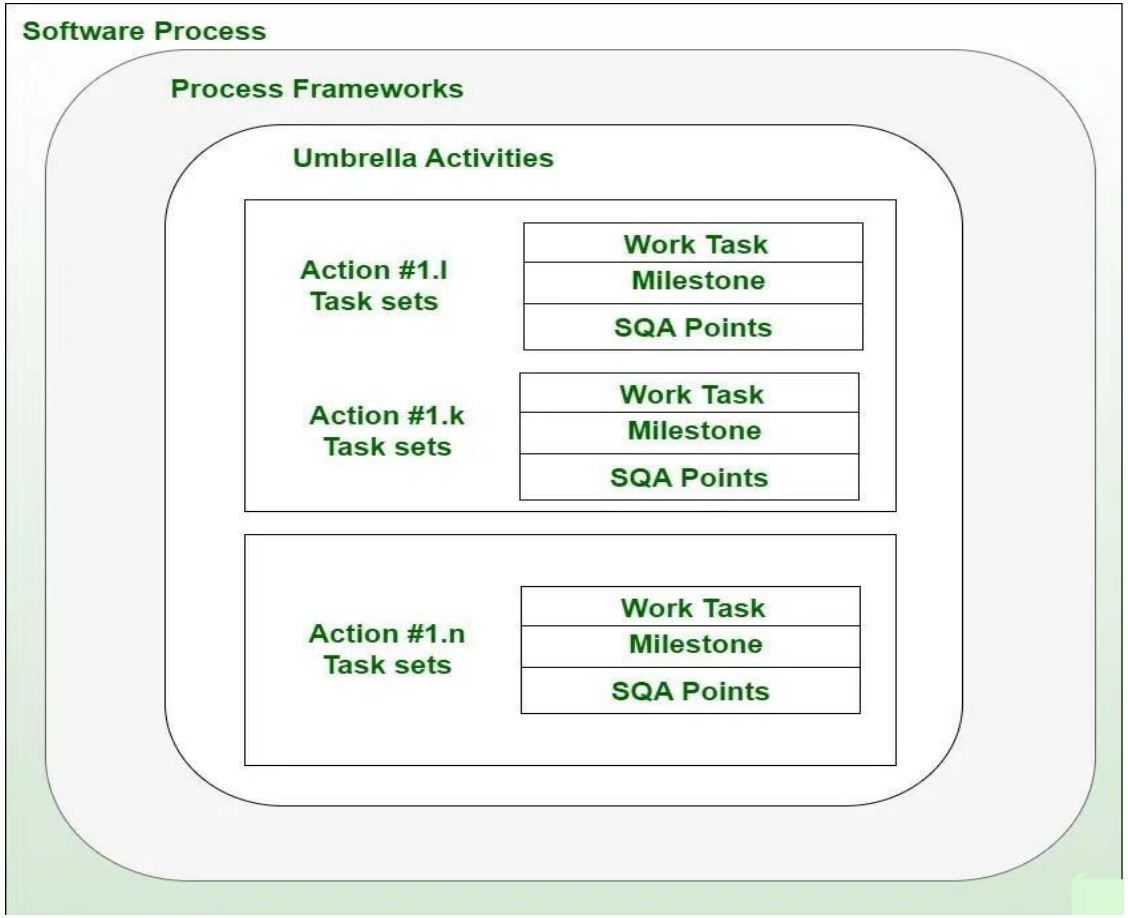
Goals of Software Process Model

- The goal of software process model is to **provide guidance for systematically coordinating and controlling the tasks** that must be performed in order to achieve end products and product objectives.

Software Process Framework

- Software Process Framework details the steps and chronological order of a process.





Software Development Life Cycle(SDLC)

- SDLC is a process which defines the various stages involved in the development of software for delivering a high-quality product.
- SDLC stages cover the complete life cycle of a software i.e. from inception to retirement of the product.
- Purpose of SDLC is to deliver a high-quality product which is as per the customer's requirement.



SDLC Phases

- Common phases in SDLC are:
    1. Communication, Planning and Analysis
    2. Design
    3. Implementation/Coding
    4. Testing and Integration
    5. Deployment
    6. Maintenance

Waterfall Model

- The Waterfall Model was the first Process Model to be introduced
- It is also referred to as a **linear-sequential life cycle model**.
- The waterfall model emphasizes a logical progression of steps.
- In this model, the whole process of software development is divided into separate phases and typically, the outcome of one phase acts as input for next phase.
- All the phases in this model are cascaded to each other in which progress is seen as flowing steadily downwards (like a waterfall) through the phases.
- The next phase is started only after the defined set of goals are achieved for previous phase and it is signed off, so the name "Waterfall Model".
- This means that any phase in the development process begins only if the previous phase is complete.

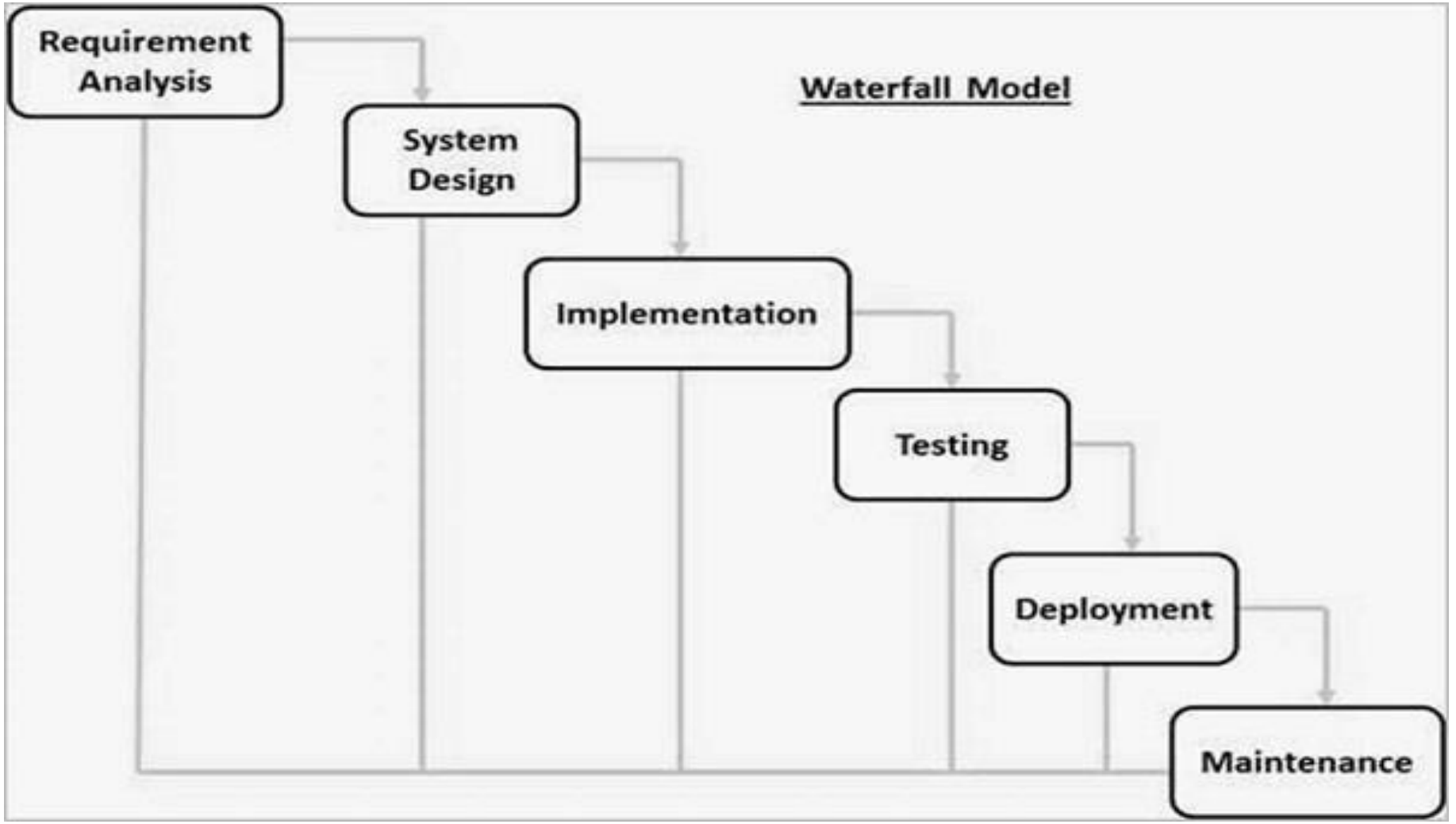


Figure: Waterfall Model

- The sequential phases of waterfall model are:

a. Requirement Gathering and analysis

- It is the first phase in the development cycle which involves which involves capturing all the possible requirements and documenting in a requirement specification document.
- Also the captured requirements are analyzed and communicated with the developing team.

b. System Design

- The requirement specifications from first phase are studied in this phase and the system design is prepared.
- This system design helps in specifying hardware and system requirements and helps in defining the overall system architecture.

c. Implementation

- The output of the system design phase is fed as input to this phase where the system is first developed in small programs called units.
- Each unit is developed as well as tested separately for its functionality.

d. Integration and Testing

- In this phase all the units are tested referred as unit testing and finally are integrated into a system.
- Again after the integration is completed the whole system is tested for faults and failures.
- This phase involves both te functional and non-functional testing.

e. Deployment of system

- Once all the testing is done the system is now ready to be deployed.
- The product is deployed in the customer environment or released into the market.

f. Maintenance

- There are some issues which come up in the client environment.
- To fix those issues, patches(set of changes to a computer program) are released. Also to enhance the product some better versions are released.
- Maintenance is done to deliver these changes in the customer environment.

All these phases are cascaded to each other in which progress is seen as flowing steadily downwards (like a waterfall) through the phases. The next phase is started only after the defined set of goals are achieved for previous phase. In this model phases do not overlap.

When is waterfall model appropriate?

Waterfall model is appropriate when:

- Product definition is stable.
- Technology is understood and is not dynamic.
- There are no ambiguous requirements i.e. when the requirements are clear, well documented and fixed.
- Sufficient resources with required expertise are available to support the product.

Advantages of Waterfall Model

Some of the major advantages of the Waterfall Model are as follows:

- The waterfall model progresses through easily understandable and explainable phases and thus it is easy to use.
- Easy to manage due to the rigidity of the model. Each phase has specific deliverables and a review process.
- Phases are processed and completed one at a time.
- Works well for smaller projects where requirements are very well understood.
- In this model, phases are processed and completed one at a time and they do not overlap.
- It is easy to arrange tasks in waterfall model.
- Process and results are well documented.

Disadvantages of Waterfall Model

- It does not allow much reflection or revision. Once an application is in the testing stage, it is very difficult to go back and change something that was not well-documented or thought upon in the concept stage.
- No working software is produced until late during the life cycle.
- High amounts of risk and uncertainty. Not suitable for projects whose requirements change frequently.
- Not a good model for complex and object-oriented projects. Poor model for long and ongoing projects..
- It is difficult to measure progress within stages.
- Cannot accommodate changing requirements.Adjusting scope during the life cycle can end a project.
- Integration is done as a big-bang at the very end, which doesn't allow identifying any technological or business bottleneck or challenges early.

Why waterfall model failed?

a. **Overlapping:** The waterfall model **has lacked an overlapping among phase.** The waterfall model recommends that new phase can start only after the completion of the previous phase. But in real projects, this can't be maintained.

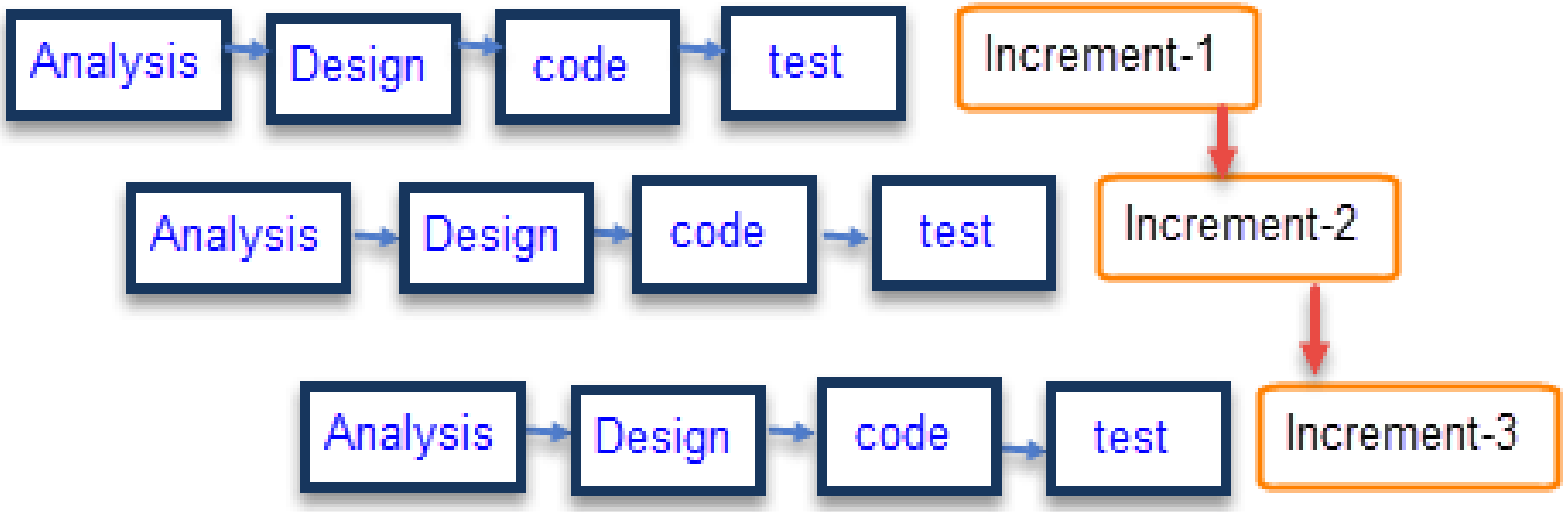
b. **Single way mechanism:** This model is just **like the one-way street**. Once a phase is completed and next phase has started then **there is no way to go back** on the previous phase.

c. **Not Flexible: Difficult to accommodate change requests**. The waterfall model assumes that all the customer requirements can be completely and correctly defined at the beginning of the project, but actually customers' requirements keep on changing with time.

d. **Lack of interaction:** The waterfall model **has lacked interaction among phase.** Feedback is not taken during development. After a development process starts, changes can not accommodate easily.

Incremental Development Model

- Incremental: a little more is added **each time/adding new functionality in small chunks**
- It is a type of evolutionary model.
- Incremental Model is a process of software development where requirements are broken down into multiple standalone modules of software development cycle.
- Incremental development is done in steps from analysis design, implementation, testing/verification, maintenance.
- First, a simple working system implementing only a few basic features is built and then that is delivered to the customer. Then thereafter many successive iterations/ versions are implemented and delivered to the customer until the desired system is realized.
- As each successive version of the software is constructed and delivered, now the feedback of the Customer is to be taken and these were then incorporated in the next version.



Incremental Model

Incremental Development Model

Pros:

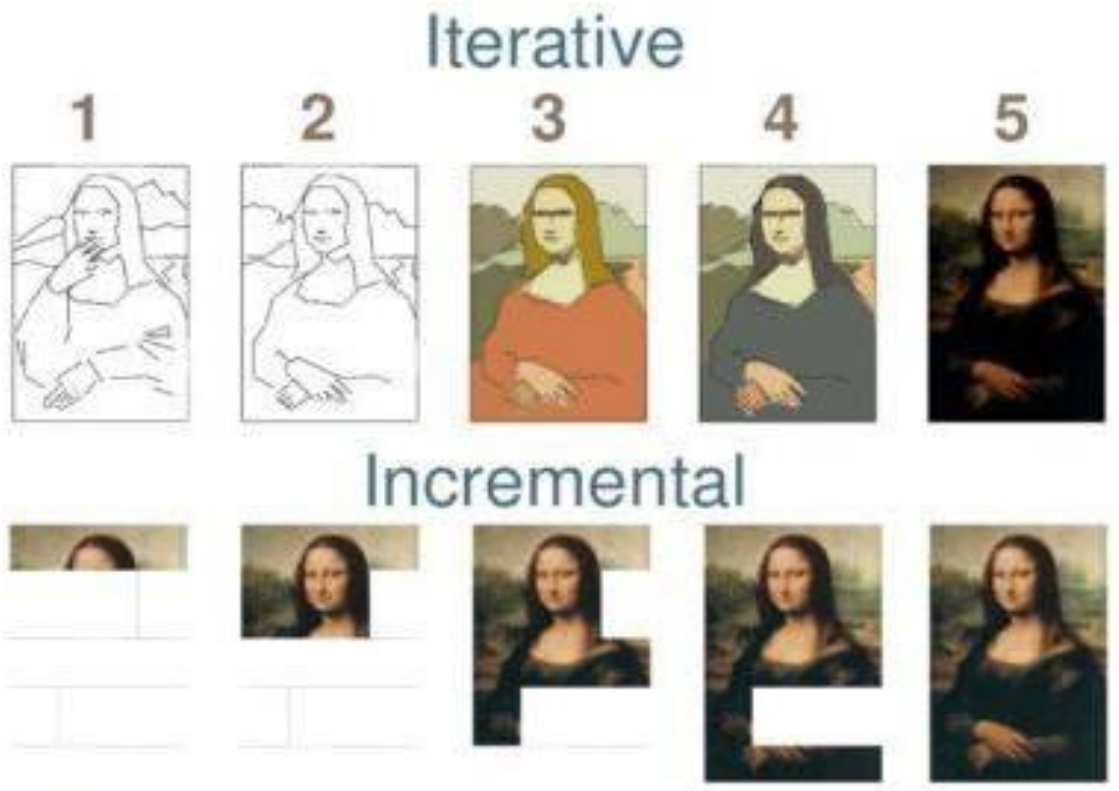
1. Generates working software quickly and early during the software life cycle.
2. More flexible – less costly to change scope and requirements.
3. Easier to test and debug during a smaller iteration.
4. Easier to manage risk because risky pieces are identified and handled during its iteration.
5. Each iteration is an easily managed milestone.

Incremental Development Model

Cons:

1. It requires a good planning designing.
2. It is costlier than waterfall model.
3. Definition of system should be complete and clear.
4. **Increased complexity:** As the project is developed incrementally, it can become more complex as each increment is added. This can make it harder to manage and maintain, as well as increase the risk of errors and bugs.
5. **Higher costs:** Since each increment requires its own planning, design, coding, testing, and deployment, the overall cost of the project can be higher than with other development methodologies.
6. **Difficulty in tracking progress:** With multiple increments being developed simultaneously, it can be challenging to track the progress of the project as a whole. This can make it harder to identify potential issues early on and take corrective action.
7. **Increased communication overhead:** With each increment being developed by a different team or individual, there can be a significant increase in the communication overhead required to ensure that everyone is on the same page.

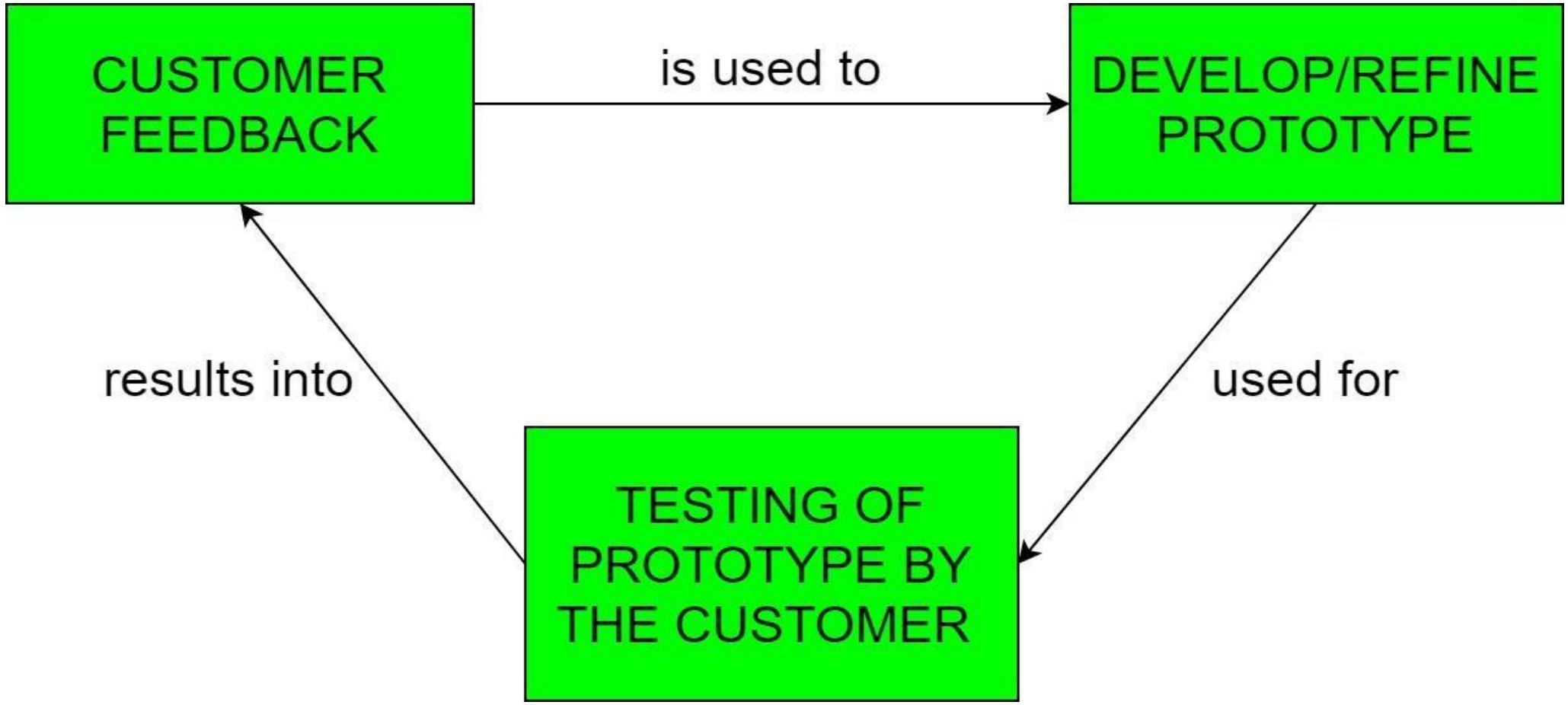
Iterative vs Incremental Approach



Prototyping Model

- Prototype is a working model of software with some limited functionality.
- **Prototyping Model** is a software development model in which prototype is built, tested, and reworked until an acceptable prototype is achieved.
- This model is used when the customers do not know the exact project requirements beforehand.
- A prototype of the end product is first developed, tested and refined as per customer feedback repeatedly till a final acceptable prototype is achieved which forms the basis for developing the final product.

Prototyping Model



Prototyping Model

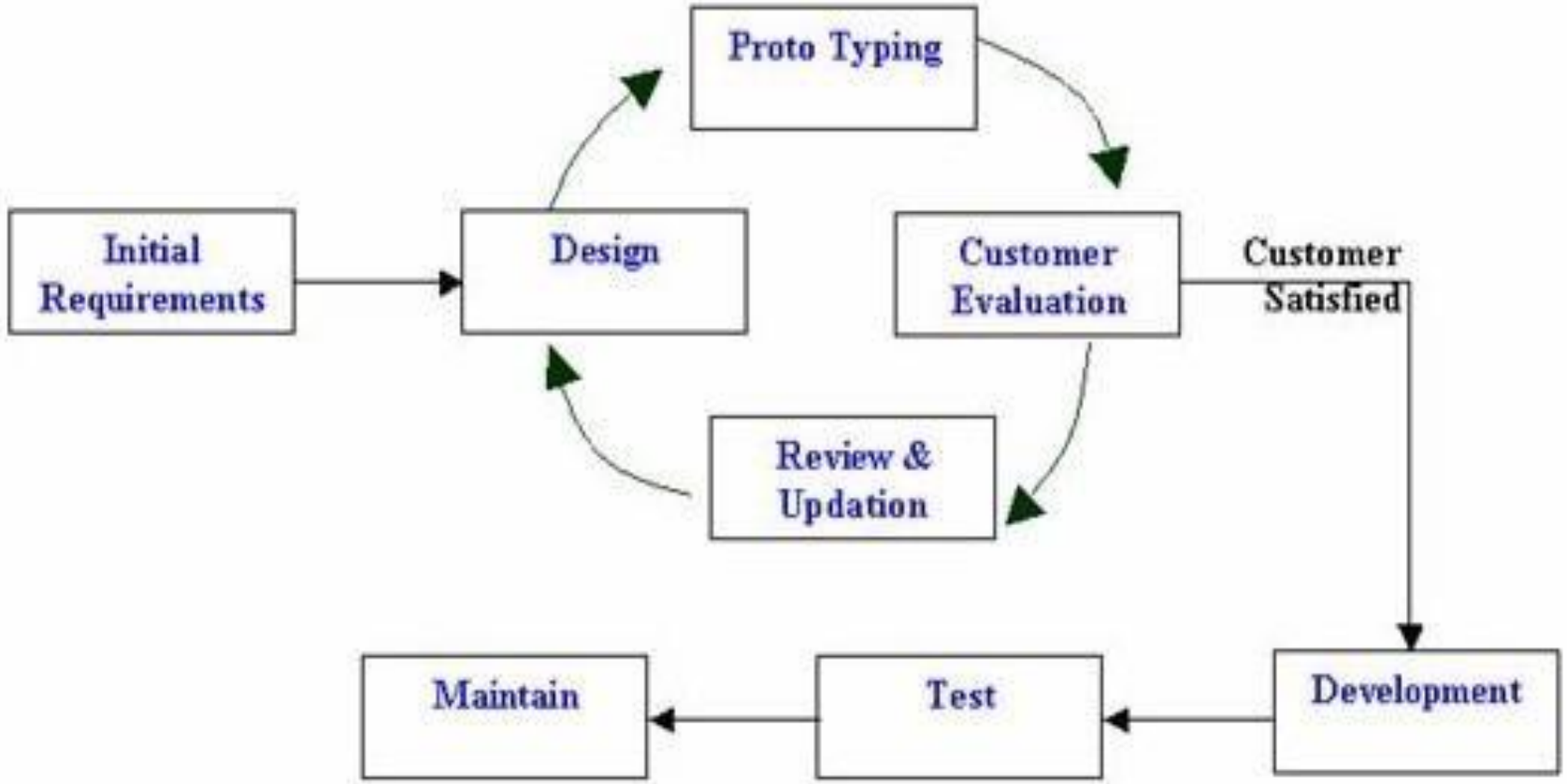


Figure: Prototyping Model

Prototyping Model-When is prototyping model suitable?

- When the customers do not know the exact project requirements beforehand.
- Prototype model should be used when the desired system needs to have a lot of interaction with the end users.
- Excellent for designing good human computer interface systems.

Pros and Cons of Prototyping Model

Pros:

1. Increased user involvement in the product even before its implementation.
2. Since a working model of the system is displayed, the users get a better understanding of the system being developed.
3. Reduces time and cost as the defects can be detected much earlier.
4. Quicker user feedback is available leading to better solutions.
5. Missing functionality can be identified easily.
6. Confusing or difficult functions can be identified.

Pros and Cons of Prototyping Model

Cons:

1. Users may get confused in the prototypes and actual systems.
2. Practically, this methodology may increase the complexity of the system as scope of the system may expand beyond original plans.
3. Developers may try to reuse the existing prototypes to build the actual system, even when it is not technically feasible.
4. The effort invested in building prototypes may be too much if it is not monitored properly

Spiral Development Model

- Spiral model is a combination of iterative development process model and sequential linear development model i.e. the waterfall model with a very high emphasis on risk analysis.
- This model is best used for large projects which involves continuous enhancements.
- There are specific activities which are done in one iteration (spiral) where the output is a small prototype of the large software.
- The same activities are then repeated for all the spirals till the entire software is build.
- Each loop of the spiral is called a Phase of the software development process.
- The exact number of phases needed to develop the product depends upon the project risk.

Spiral Development Model

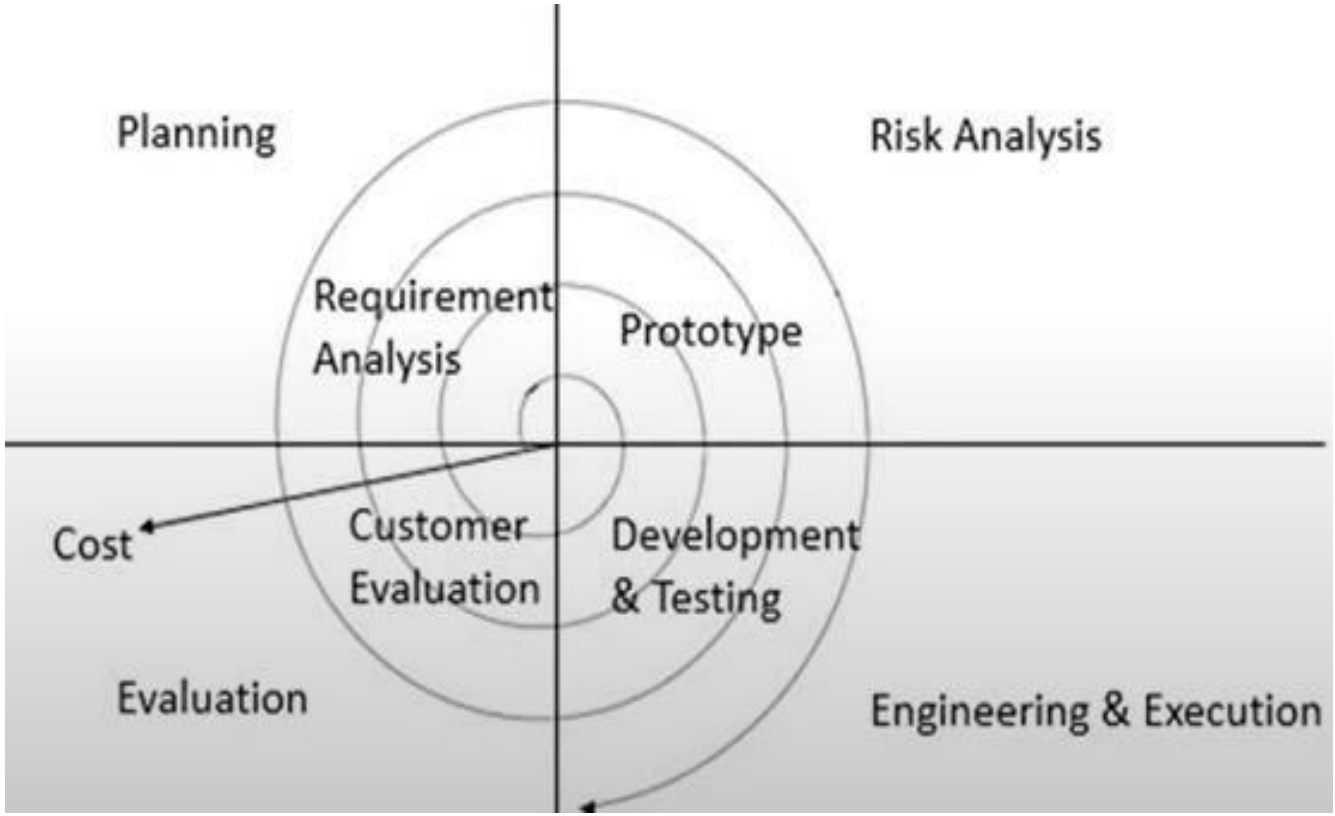


Figure: Spiral Model

Spiral Development Model

- Spiral model, generally involves four phases, which are followed repeatedly, in each round of the model, until no further requirements are needed to be implemented in the software product.
- The **radius of the spiral** at any point **represents the expenses(cost)** of the project so far, and the **angular dimension** represents the **progress made** so far in the current phase.

When is Spiral Model applicable?

- When there is a budget constraint and risk evaluation is important.
- For medium to high-risk projects.
- Long-term project commitment because of potential changes to economic priorities as the requirements change with time.
- Customer is not sure of their requirements which is usually the case.
- Requirements are complex and need evaluation to get clarity.
- New product line which should be released in phases to get enough customer feedback.
- Significant changes are expected in the product during the development cycle.

Pros and Cons of Spiral Model

Pros:

1. Changing requirements can be accommodated.
2. Allows extensive use of prototypes.
3. Requirements can be captured more accurately.
4. Users see the system early.
5. Development can be divided into smaller parts and the risky parts can be developed earlier which helps in better risk management.

Pros and Cons of Spiral Model

Cons:

1. Management is more complex. Risk analysis requires high expertise.
2. End of the project may not be known early.
3. Not suitable for small or low risk projects and could be expensive for small projects.
4. Process is complex
5. Spiral may go on indefinitely.
6. Large number of intermediate stages requires excessive documentation.

Software Requirements

- The software requirements are description of features and functionalities of the target system.
- According to IEEE standard 729, a requirement is defined as follows:
  - A condition or capability needed by a user to solve a problem or achieve an objective
  - A condition or capability that must be met or possessed by a system or system component to satisfy a contract, standard, specification or other formally imposed documents
- The requirements can be obvious or hidden, known or unknown, expected or unexpected from clients point of view.

Types of Requirements

1. **Functional Requirements**
2. **Non-Functional Requirements**
3. Domain Requirements

Types of Requirements-Functional Requirements

- Functional requirements are those requirements that deal with what the system should do or provide for the users.
- Any requirement which specifies what the system should do.
- Functional requirements should address:
    a. what the system should provide
    b. how the system should react to particular inputs
    c. how the system should react in particular situations.
- Examples:
    a. **User Authentication**: The system must allow users to log in using a username and password.
    b. **Search Functionality**: The software should enable users to search for products by name or category.
    c. **Report Generation**: The system should be able to generate sales reports for a specified date range.

Types of Requirements-Functional Requirements

- Non-Functional Requirements(NFR) are those requirement that specifies criteria that can be used to judge the operation of a system, rather than specific behaviors.
- Non-functional requirements generally specify the system's quality attributes or characteristics.
- Non-functional requirements are often called "quality attributes" of a system.
- Non-functional requirements can be divided into two main categories:
  a. **Execution qualities**, such as safety, security and usability, which are observable during operation (at run time).
  b. **Evolution qualities,** such as testability, maintainability, extensibility and scalability, which are embodied in the static structure of the system.
- **Examples**:
  - **Performance**: The system should process 1,000 transactions per second.
  - **Usability**: The software should be easy to use and have a user-friendly interface.
  - **Reliability**: The system must have 99.9% uptime.
  - **Security**: Data must be encrypted during transmission and storage.

Non-Functional Requirement

- Some Non-Functional Requirements are:

1. Availability: It is the time frame in which the system functions normally and without failures.
2. Efficiency: It is the system capability to management resources as CPU, Memory and Network Infrastructure
3. Performance: It is the system capability to do specific actions in a given time span.
4. Reliability: It is measured as the probability that the software will not fail.
5. Robustness: The ability of the system to control and continue to work with exceptions, unexpected conditions without affecting the user's experience.
6. Accuracy: System capacity to perform calculations or algorithms with a certain precision

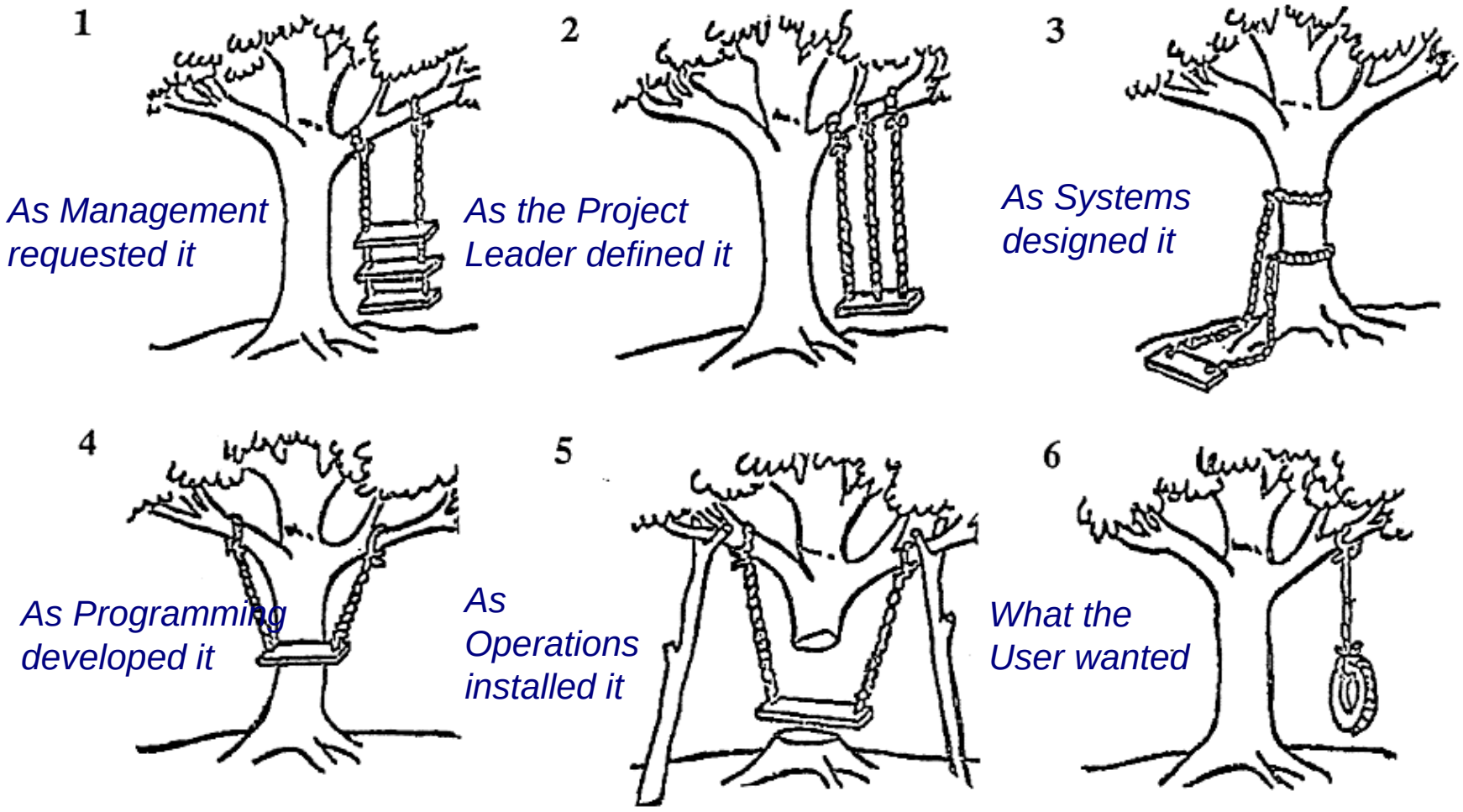
Non-Functional Requirement

7. Security: System capacity to avoid malicious incidents or activities. Its must keep in mind the Access, Privacy, Integrity and Audit of its information.
8. Interoperability: It is the system capability to communicate successfully with different external systems.
9. Maintainability: How fast the system can be maintained or changes can be made.
10. Extensibility: It is the system capability to easily add new modules in a planned way.
11. Manageability: It is the capability of the system to monitor and manage the system by administrators.
12. Portability: It is the ability of the system to run on different computing platforms.
13. Reusability: It is the capacity that defines the degree to which part of the system can be re-used.
14. Usability: Define the degree of ease with which the user can learn and operate

Functional vs Non-Functional Requirements

Functional Requirements	Non-Functional requirements
Functional requirements are those requirements that deal with what the system should do or provide for the users.	Non-Functional Requirements(NFR) are those requirement that specifies criteria that can be used to judge the operation of a system, rather than specific behaviors.
Describe the action with which the user's work is connected.	Describe the experience of the user while doing the work.
These are the functions that can be captured on use case.	These are the global constraints on software system.
Functional Requirements are mandatory	Non-Functional Requirements are not mandatory.
These requirements are easy to capture.	Hard to capture.
Involves Functional Testing like System, Integration, End to End, API testing, etc.	Involves Non-Functional Testing like Performance, Stress, Usability, Security testing, etc.
Example: Consider a flight reservation system, where a user can search for a flight. If a flight is selected, the user has 2 minutes to confirm the booking or the flight will be released	Example: For the flight reservation system, the response time for a search transaction under a peak load of 10,000 users must not exceed 2 seconds

Requirements Engineering-Impedance Mismatches



Requirement Engineering

- Requirement Engineering is the process of defining, documenting and maintaining the requirements.
- Requirements engineering is a process of gathering and defining of what the services should be provided by the system.
- The goal of requirement engineering is to develop and maintain sophisticated and descriptive ‘System Requirements Specification’ document.
- Requirement engineering provides the appropriate mechanism to:
    1. understand what the customer desires
    2. analyzing the need, and assessing feasibility
    3. negotiating a reasonable solution
    4. specifying the solution clearly
    5. validating the specifications and managing the requirements

Requirement Engineering Process Activities

1. Feasibility Study
2. Requirements Elicitation and Analysis
3. Requirements Validation
4. Requirements Management

1. Feasibility Study

- A feasibility study decides whether or not the proposed system is worthwhile.
- During Feasibility Study, it is studied whether the desired system and its functionality are feasible to develop.
- Feasibility study is focused towards goal of the organization.
- A short focused study that checks:
  - If the system contributes to organizational objectives
  - If the system can be engineered using current technology and within budget
  - If the system can be integrated with other systems that are used
- The output of this phase should be a feasibility study report that should contain adequate comments and recommendations for management about whether or not the project should be undertaken.

Types of Feasibility Study

1. Technical Feasibility

- In Technical Feasibility current resources both hardware software along with required technology are analyzed/assessed to develop project.
- This technical feasibility study gives report whether there exists correct required resources and technologies which will be used for project development.
- Along with this, technical feasibility study also analyzes:

- Technical skills and capabilities of technical team
- Existing technology can be used or not
- Maintenance and up-gradation is easy or not for chosen technology etc.

Types of Feasibility Study

2. Economic Feasibility

- This assessment typically involves:
  - a cost/ benefits analysis of the project
  - helping organizations determine the viability, cost, and benefits associated with a project before financial resources are allocated.
- After that it is analyzed whether project will be beneficial in terms of finance for organization or not.

Types of Feasibility Study

3. Operational Feasibility

- This assessment involves undertaking a study to analyze and determine whether and how well the organization's needs can be met by completing the project.
- Degree of providing service to requirements is analyzed along with how much easy product will be to operate and maintain.
- Determines suggested solution by software development team is acceptable or not

Types of Feasibility Study

4. Legal Feasibility

- In Legal Feasibility study, project is analyzed in legality point of view.
- This includes analyzing barriers of legal implementation of project, data protection acts or social media laws, project certificate, license, copyright etc.
- Overall it can be said that Legal Feasibility Study is study to know if proposed project conform legal and ethical requirements.

Types of Feasibility Study

5. Schedule Feasibility

- In Schedule Feasibility Study, mainly timelines/deadlines is analyzed for proposed project.
- It includes how many times teams will take to complete final project which has a great impact on the organization as purpose of project may fail if it can't be completed on time.

Key benefits of conducting a Feasibility Study

- Improves project team's focus
- Identifies new opportunities
- Provides valuable information for a "go/no-go" decision
- Narrows the business alternatives
- Identifies a valid reason to undertake the project
- Enhances the success rate by evaluating multiple parameters
- Aids decision-making on the project
- Identifies reasons not to proceed

2. Requirement Elicitation and Analysis

- Requirements Elicitation is the first phase in requirements engineering
- Requirements Elicitation is the practice of researching and discovering the requirements of a system.
- Requirement Elicitation is also sometimes referred to as "requirement gathering"
- Requirement Elicitation and Analysis is a process of interacting with customers and end-users to find out about the domain requirements, what services the system should provide, and the other constraints
- There are four main activities during requirement elicitation and analysis.

1. Requirements Discovery
2. Requirements Classification and Organization
3. Requirements Prioritization and Negotiation
4. Requirements Specification

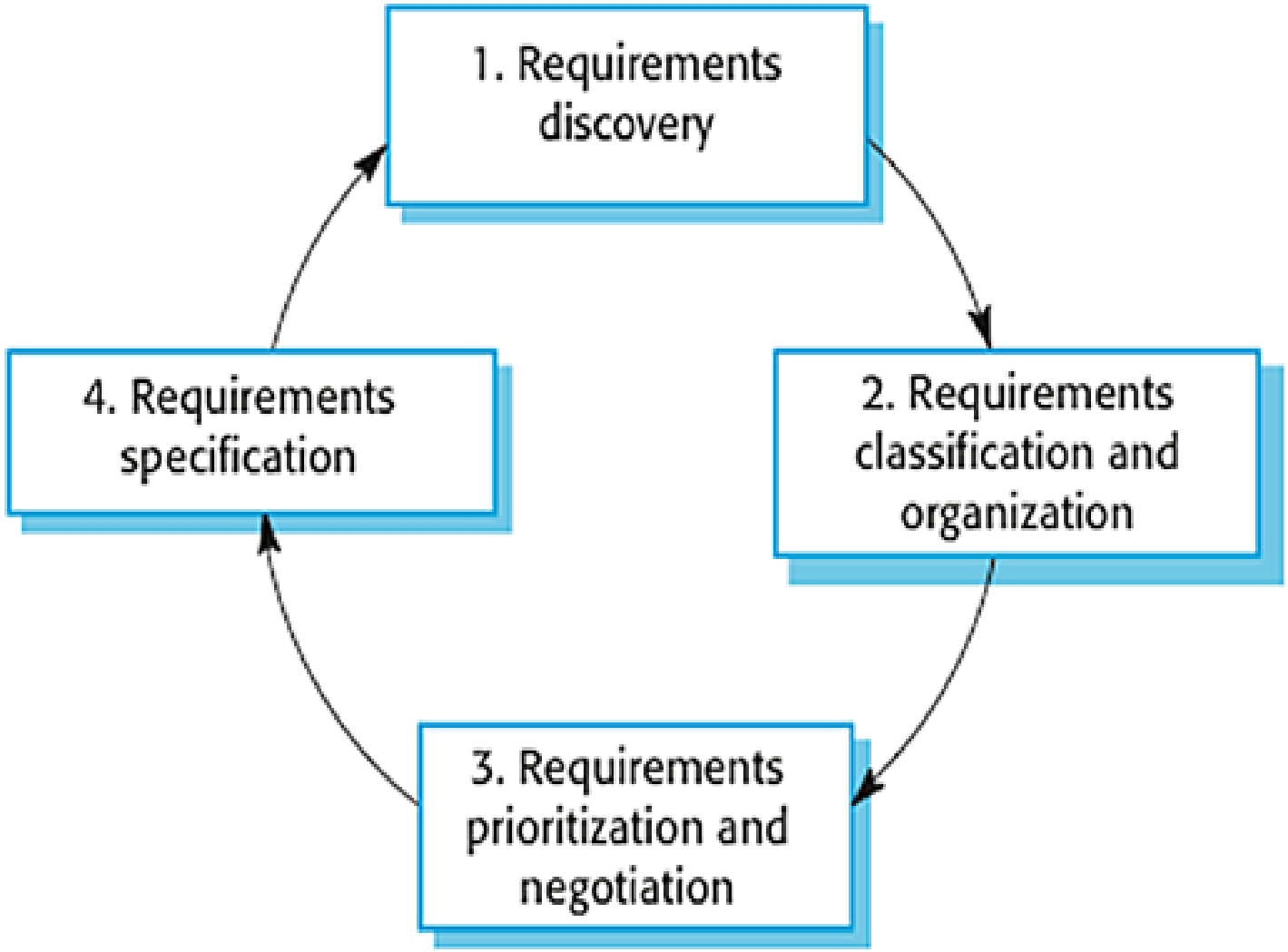


Figure: The process of requirements elicitation and analysis

2.1. Requirements Discovery

- It's the process of interacting with, and gathering the requirements from, the stakeholders about the required system and the existing system
- Gathering and understanding the requirements is a difficult process
- It can be done using some techniques like:

a. Interviews
b. Brainstorming Sessions
c. Facilitated Application Specification Technique (FAST)
d. Quality Function Deployment (QFD)
e. Use Cases and Scenarios
f. Requirements reuse
g. Observations and social analysis

These techniques are also known as requirement elicitation techniques

a. Interview

- Objective of conducting an interview is to understand the customer's expectations from the software.
- It is impossible to interview every stakeholder hence representatives from groups are selected based on their expertise and credibility.
- Interviews maybe be open ended or structured.

1. In open ended interviews there is no pre-set agenda. Context free questions may be asked to understand the problem.
2. In structured interview, agenda of fairly open questions is prepared. Sometimes a proper questionnaire is designed for the interview.

- In practice, interviews usually use mixture of both.
- Usually, start with the open-ended questions, and then use the closed-ended questions to be more specific about some requirements

b. Brainstorming Sessions

- It is a group technique
- It is intended to generate lots of new ideas hence providing a platform to share views
- A highly trained facilitator is required to handle group bias and group conflicts.
- Every idea is documented so that everyone can see it.
- Finally a document is prepared which consists of the list of requirements and their priority if possible.

c. Quality Function Deployment

- In this technique customer satisfaction is of prime concern.
- It emphasizes on the requirements which are valuable to the customer.
- 3 types of requirements are identified:

- **Normal requirements:** In normal requirements, the objective and goals of the proposed software are discussed with the customer
- **Expected requirements:** These requirements are so obvious that the customer need not explicitly state them.
- **Exciting requirements:** These requirements include features that are beyond customer's expectations and prove to be very satisfying when present.

d. Use Cases and Scenarios

- Use cases identify interactions between the system and it's users or even other external systems.
- A scenario is a textual description of one or more of these interactions.
- This technique combines text and picScenarios are descriptions of how a system is used in practice.
- The use cases describe the 'what', of a system and not 'how'. Hence they only give a functional view of the system.
- Use case and scenarios are effective techniques for eliciting the requirements.
- Since they focus on the interactions with the system, they aren't effective for eliciting high-level business, non-functional, or domain requirements.

e. Requirements Reuse

- Reuse involves taking the requirements which have been developed for one system and using them in a different system.
- Requirements reuse saves time and effort as reused requirements have already been analyzed and validated in other systems

f. Observations and social analysis

- Requirements can be gathered from observing existing systems as well as via social analysis.

2.2. Requirements Classification and Organization

- It is the second phase in requirements elicitation and analysis.
- The requirements gathered in first phase are classified and organized in this phase.
- It involves putting related requirements together, and decomposing the system into sub components of related requirements.
- Then, we define the relationship between these components.

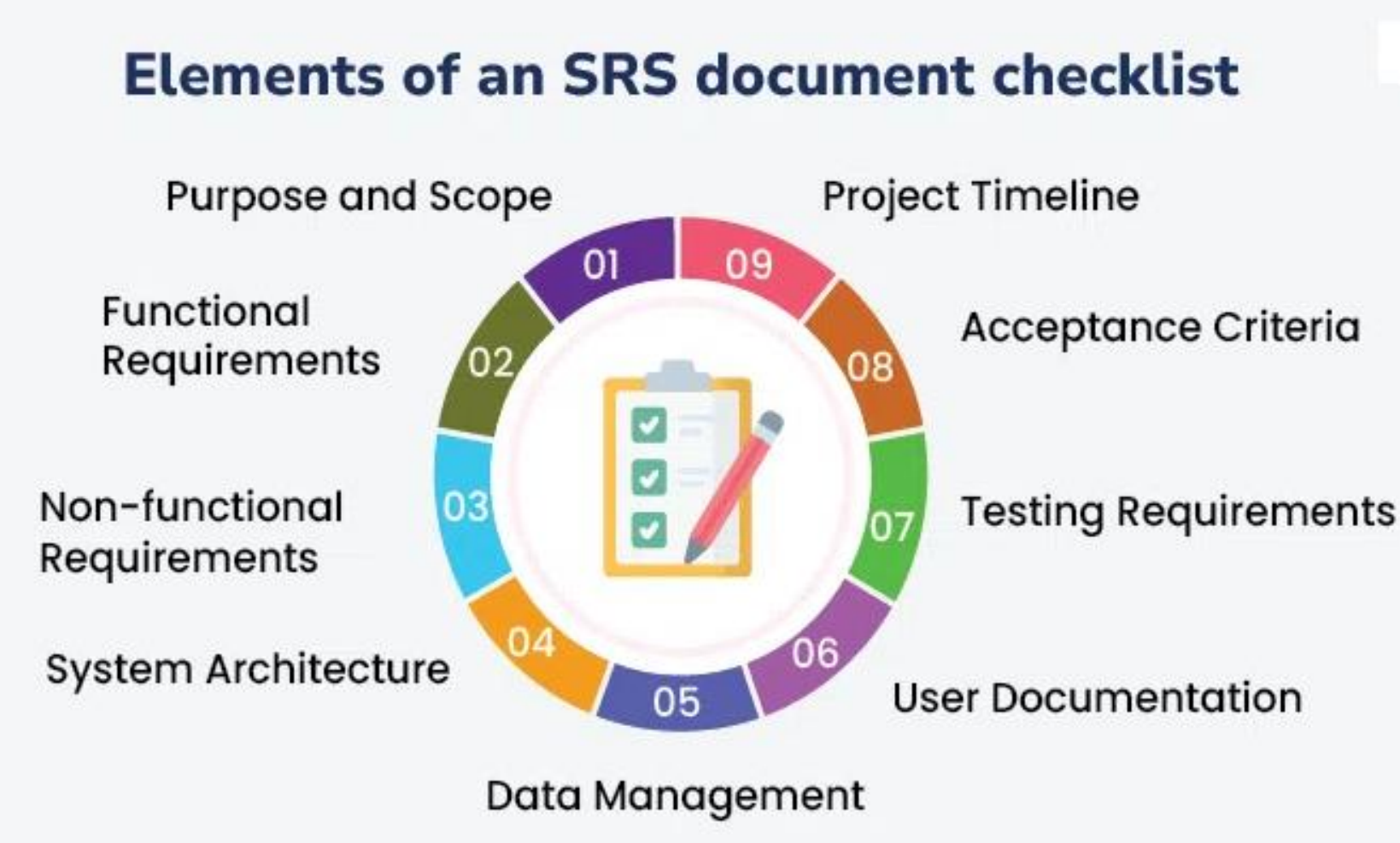
2.3 Requirements Prioritization and Negotiation

- This activity is concerned with prioritizing requirements and finding and resolving requirements conflicts through negotiations until we reach a situation where some of the stakeholders can compromise.
- We shouldn't reach a situation where a stakeholder is not satisfied because his requirements is not taken into consideration.
- Prioritizing your requirements will help us later to focus on the essentials and core features of the system, so we can meet the user expectations.
- It can be achieved by giving every piece of function a priority level.

2.4 Requirements Specification

- It’s the process of writing down the user and system requirements into a document.
- A detailed software description which can serve as a basis for a design or implementation.
- A Software Requirements Specification (SRS) is a detailed description of a software system to be developed with its functional and non-functional requirements.
- An SRS minimizes the time and effort required by developers to achieve desired goals and also minimizes the development cost.
- It may include the use cases of how user is going to interact with software system.
- The requirements should be clear, easy to understand, complete and consistent.
- In practice, this is difficult to achieve as stakeholders interpret the requirements in different ways and there are often inherent conflicts and inconsistencies in the requirements.

System/Software Requirements Specification(SRS)



System/Software Requirements Specification(SRS)

Before implementing SRS following things need to be considered:

1. Correctness of SRS should be checked.
2. Ambiguity should be avoided
3. Requirements should be complete
4. Consistent requirements
5. Verification of expected result
6. Testing environment
7. Pre-conditions defined clearly
8. Security and Performance criteria
9. Assumption should be avoided
10. Deletion of irrelevant requirements

3. Requirements Validation

- Requirement Validation is the process of ensuring the specified requirements meet the customer needs.
- Requirements validation is the process of checking that requirements defined for development, define the system that the customer really wants.
- To check issues related to requirements, we perform requirements validation.
- We usually use requirements validation to check error at the initial phase of development as the error may increase excessive rework when detected later in the development process.
- In the requirements validation process, we perform a different type of test to check the requirements mentioned in the Software Requirements Specification (SRS)

3. Requirements Validation

➢These checks include:

1. **Validity checks:** The functions proposed by stakeholders should be aligned with what the system needs to perform. We may find later that there are additional or different functions are required instead.
2. **Consistency checks:** Requirements in the document shouldn't conflict or different description of the same function
3. **Completeness checks:** The document should include all the requirements and constraints.
4. **Realism checks:** Ensure the requirements can actually be implemented using the knowledge of existing technology, the budget, schedule, etc.
5. **Verifiability:** Requirements should be written so that they can be tested. This means you should be able to write a set of tests that demonstrate that the system meets the specified requirements.
6. **Ambiguity checks:** Testing for ambiguity/ambiguous requirements

Techniques used to Validate Requirements

1. **Requirements Reviews**
   - The SRS is carefully reviewed by a group of people including people from both the organization and the client side
   - The reviewer systematically analyses the document to check error and ambiguity.
2. **Prototyping**
   - In this approach to validation, an executable model of the system is demonstrated to the customer and end users to validate, and ensure if it meets their needs.
   - Prototyping is usually used when the requirements aren’t clear.
3. **Test case generation**
   - Requirement mentioned in SRS document should be testable, the conducted tests reveal the error present in the requirement.
   - It is generally believed that if the test is difficult or impossible to design than, this usually means that requirement will be difficult to implement and it should be reconsidered.

4. **Automated Consistency Analysis**

- This approach is used for automatic detection of an error
- First, the requirement is structured in formal notation then CASE tool is used to check in-consistency of the system.
- The report of all inconsistencies is identified and corrective actions are taken.

5. **Walk-through**

- A walkthrough does not have a formally defined procedure and does not require a differentiated role assignment.
- Basically walkthrough here involves:

a. Checking early whether the idea is feasible or not.
b. Obtaining the opinions and suggestion of other people.
c. Checking the approval of others and reaching an agreement.

4. Requirements Management

- Requirements management is the process of collecting, analyzing, refining, and prioritizing product requirements and then planning for their delivery.
- The purpose of requirements management is to ensure that the organization validates and meets the needs of its customers and external and internal stakeholders.
- The purpose of requirements management is to assure the organization documents verifies and meets the needs and expectations of its customers and internal or external stakeholders.
- The essential activities performed in requirements management are listed below.
  - Recognizing the need for change in the requirements
  - Establishing a relationship amongst stakeholders and involving them in the requirements engineering process
  - Identifying and tracking requirements attributes.

Requirements Change - Why requirements change?

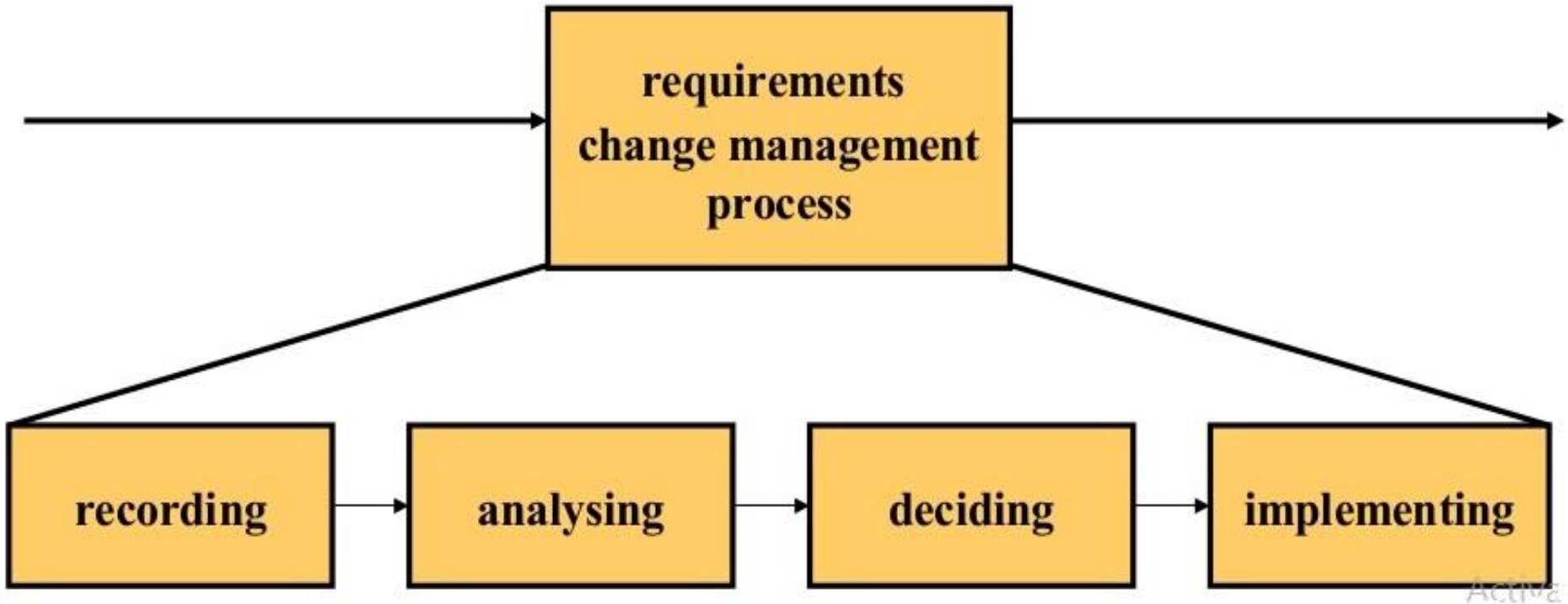
- Requirements Change is one of the critical problems faced in software development

1. **Poorly Defined Requirement Development Process**
   - A major reason for change is a poorly defined or ignored requirement development process.
   - This can result in defective requirements, incorrect requirements, and missing requirements.
   - The further in the development life cycle we go before discovering these problems, the greater the impact on cost and schedule.
   - At this situation, developers find major problems and issues with the requirements and so the changes begin
2. **Ignored Requirements**
   - The developers could have ignored some of the original requirements.
   - Either they didn't understand the requirements or the requirements were too complex or the developer just chose to ignore the requirements.

Requirements Change - Why requirements change?

3. **Immature Technology**
   - Use of immature technology during software development also results in requirements change.
4. **Need Changes**
   - Sometimes the problem that product is going to address gets changed and the basic needs of our project might change.
   - If so happens, the whole project needs to be rethought.
5. **Missing Stakeholders**
   - Key stakeholders were left out at the beginning of the project.
   - Stakeholder represent requirements.
   - Missing stakeholders result in missing requirements.
6. **Overly Optimistic budget and schedule**
   - Promising something that cannot be delivered with the available resources or in the time allotted.

Requirements Change Management



Requirements Change Management Process

1. Recording a change proposal: In this process, Stakeholders submit proposals for requirements change using a pre-defined form.
2. Analyzing a change request: The proposal submitted by Stakeholders is analyzed to see:
    - if it is valid. Sometimes stakeholders misunderstand the requirements and suggest unnecessary changes.
    - how many requirements and system components are affected.
    - How much it will cost to make change in both time and money
    - What are the benefits of the change
3. Deciding on a change request: Change requests are considered formally by a group or a person who decide whether or not the change should be accepted.
4. Implementing a change: It involves updating the requirements document and informing concerned parties about the update.

Importance of Requirement Engineering

1. Clear Understanding of User Needs
2. Improves Product Quality
3. Mitigates Risks
4. Saves Time and Cost
5. Improves Communication
6. Provides a Basis for Design and Development
7. Enhances Quality of the Final Product
8. Helps in Project Planning and Management